

Schema, Intent, and Metadata

Before the machine generates prose, it records the conditions under which that prose should be produced. In this system, the work object is not just a container for text; it is a declaration of intent. Capturing purpose, audience, authorial voice, metadata, publication settings, and epistemic stance up front gives the runtime a stable target for every downstream decision, from outline normalization to section drafting, citation routing, and artifact assembly.

Purpose answers the most basic question: what is this document trying to do? A technical paper that explains an architecture requires different treatment than a tutorial, a proposal, or a reflective essay. Audience narrows that purpose into a practical reading model. A manuscript aimed at researchers and software engineers can assume familiarity with systems terminology, but it still needs to explain the design in concrete operational terms. Authorial voice then constrains how the system should speak: whether it should sound exploratory, prescriptive, formal, or conversational. In this paper, the voice is that of a system designer explaining the machine from the inside, so the prose should be explanatory and implementation-aware rather than detached or promotional.

Metadata makes the work machine-readable and durable. Fields such as title, version, language, status, authorship, and maintenance responsibility allow the repository to track the document as a living artifact rather than a static file. Publication settings extend that control to the output side: whether a table of contents is enabled, whether front matter is included, and how the compiled artifact should be assembled. These settings matter because the same content can be rendered into different forms, and the system needs to know which form is intended before it starts generating sections.

The epistemic stance is equally important. A system can only write responsibly if it knows what kind of claims it is allowed to make. This paper is grounded in constructive systems explanation: it describes the machine by reference to working software, durable logs, and generated artifacts. That stance discourages vague speculation and encourages claims that can be traced to the repository itself. It also clarifies the role of evidence. When the manuscript describes task queues, callbacks, or artifact lifecycles, it is not inventing an abstract theory and then searching for examples; it is organizing observed behavior into a coherent account.

Taken together, these fields form a pre-generation contract. They reduce ambiguity before any section text is produced, so the system does not have to infer the document’s identity from prose alone. That matters because later stages depend on the same canonical work object. Section agents need to know what kind of document they are contributing to, gardeners need to know what editorial standard to enforce, and specialist agents need to know whether a claim belongs in the manuscript at all. By capturing intent and metadata first, the Codynamic Book Machine turns authorial direction into structured state, and structured state into reliable generation.

This early schema also supports consistency across revisions. If the audience, voice, or epistemic stance changes midstream, the resulting manuscript can drift in tone and argument even when individual sections are locally correct. By keeping those settings explicit and centralized, the system makes such drift visible and correctable. In that sense, schema is not merely administrative overhead; it is part of the manuscript’s argumentative infrastructure. It tells the machine what kind of book it is building before the machine begins to build it.

The section is intentionally self-contained and does not require a visual to carry its argument, so no diagram is requested here.